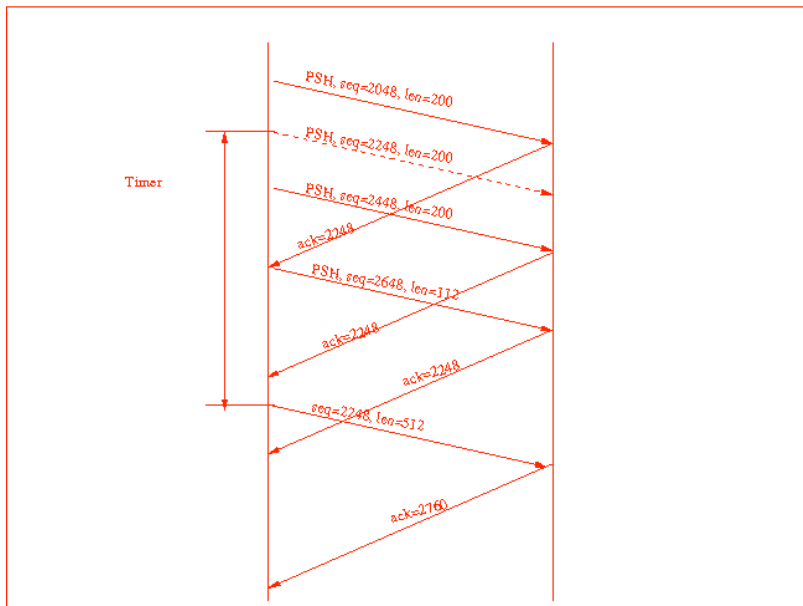
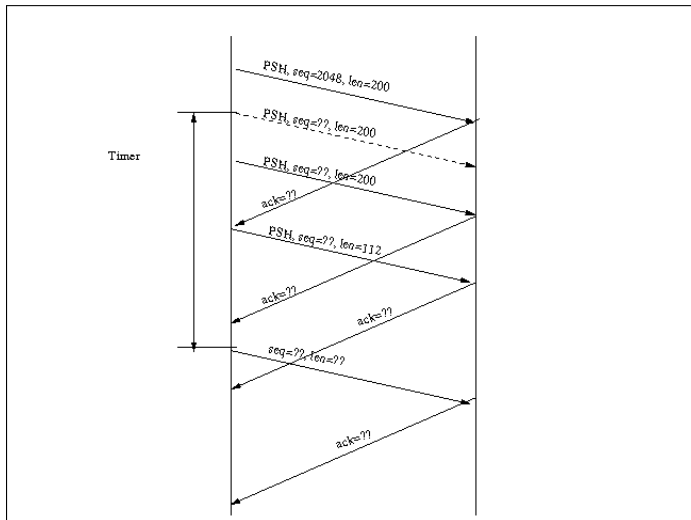


TELECOM Nancy

Module Réseau Système Avancé

Numérotation des segments

Soit une connexion TCP avec un MSS de 512 octets avec l'échange suivant que vous complétez.

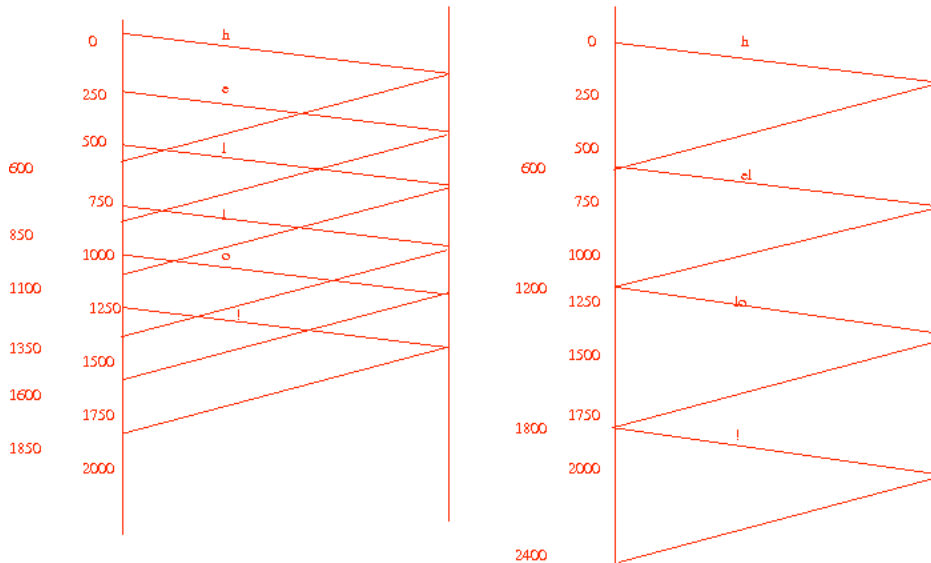


Algorithme de Nagle

L'algorithme spécifie que si une connexion a des données en attente (c'est-à-dire en attente d'acquittement) alors les petits paquets ne seront pas envoyés jusqu'à réception de l'acquittement (petit signifie < 1MSS)

- Deux applications sont génératrices de petits paquets comme telnet et rlogin. Soit par exemple, un utilisateur qui tape 6 caractères avec 250ms entre chaque caractère et un RTT de 600ms entre le client et le serveur. On fait l'hypothèse que le serveur envoie immédiatement un ACK. Représenter l'échange des données
 - sans activation de l'algorithme de Nagle
 - avec activation

Correction



- À votre avis, quel est l'intérêt d'un tel algorithme ?

L'algorithme de Nagle permet de réduire l'émission de petits paquets (et par conséquent une consommation de la bande passante). Dans le 1^{er} schéma, il y a 12 paquets en tout. Dans le 2^{ème}, seulement 8. Si les petits paquets ne sont pas un problème sur les LAN mais ils peuvent ajouter de la congestion dans les WAN (réseaux étendus).

- Pourquoi peut-il être intéressant de désactiver cet algorithme ?

À cause des délais, comme on peut le voir sur les schémas. Si on associe à l'algorithme de Nagle, l'algorithme du « delayed-ack » qui permet de ne pas retourner immédiatement un ACK (attendre entre 50 et 200ms), ces délais peuvent s'accroître.

Intéressant le « delayed-ack », si le serveur doit retourner des données. Dans le cas contraire, le client ne pourra rien envoyer tant que l'acquittement (sans données) sera revenu.

Le syndrome de la « silly window »

Ce syndrome se produit quand de petits montants de données sont échangés au lieu des segments complets. Cela peut se produire notamment quand le récepteur annonce de petites fenêtres pour des raisons de contrôles de flux.

Soit une taille de MSS de 512 et une fenêtre côté récepteur de la même valeur. Cela signifie que le client peut envoyer un segment complet au serveur. Supposons que le serveur, pour des raisons de surcharge ne peut supprimer que 1 octet de son buffer pour 4 reçus et qu'il supprime 30 octets de son buffer chaque fois qu'un segment côté client arrive.

Que va-t-il se passer au niveau de l'évolution des fenêtres.

1. La fenêtre d'émission du client est de 512 octets et il envoie immédiatement les 512 octets (une fenêtre complète)
2. Quand le serveur reçoit l'acquittement, il va supprimer du buffer $(512/4) = 128$ octets. Il va donc demander au client de réduire sa fenêtre à 128 octets. Il reste dans le buffer : $512 - 128 = 384$
3. Le client reçoit un acquittement de 512 octets et voit que sa fenêtre est réduite à 128. Il va donc envoyer le plus de données possible soit 128 octets par segment.
4. Le serveur a détruit 30 octets pendant que les 128 octets arrivent. Il reste dans le buffer $= 384 - 30 = 354$. Le serveur est capable de traiter $128/4 = 32$ octets. 96 sont ajoutés au 354 dans le buffer. Le buffer contient $354 + 96 = 450$. Le serveur doit réduire sa fenêtre de $512 - 450 = 62$
5. Le serveur a détruit 30 octets pendant que les 62 octets arrivent. Il reste dans le buffer $= 450 - 30 = 420$. Le serveur est capable de traiter $62/4 = 15$ octets. 47 sont ajoutés au 420 dans le buffer. Le buffer contient $420 + 47 = 467$. Le serveur doit réduire sa fenêtre de $512 - 467 = 45$
6. Le serveur a détruit 30 octets pendant que les 45 octets arrivent. Il reste dans le buffer $= 467 - 30 = 437$. Le serveur est capable de traiter $45/4 = 11$ octets. 34 sont ajoutés au 437 dans le buffer. Le buffer contient $437 + 34 = 471$. Le serveur doit réduire sa fenêtre de $512 - 471 = 41$
7. Le serveur a détruit 30 octets pendant que les 41 octets arrivent. Il reste dans le buffer $= 471 - 30 = 441$. Le serveur est capable de traiter $41/4 = 10$ octets. 31 sont ajoutés au 441 dans le buffer. Le buffer contient $441 + 31 = 472$. Le serveur doit réduire sa fenêtre de $512 - 472 = 40$
8. Le serveur a détruit 30 octets pendant que les 40 octets arrivent. Il reste dans le buffer $= 472 - 30 = 442$. Le serveur est capable de traiter $40/4 = 10$ octets. 30 sont ajoutés au 442 dans le buffer. Le buffer contient $442 + 30 = 472$. Le serveur doit réduire sa fenêtre de $512 - 472 = 40$
9. ... On arrive très vite à une petite taille de fenêtre.

Pour empêcher cela

- a) Côté récepteur, on met en place le mécanisme dit de "**receiver avoidance of silly window syndrome**":
 - le récepteur ne doit pas annoncer de petites fenêtres.
 - le récepteur annonce une plus large fenêtre que celle actuellement annoncée (**qui peut être 0**) si la fenêtre peut être augmentée de MIN (la taille d'un segment complet, 1/2 de l'espace buffer du récepteur) .

Annoncer une fenêtre de 0 permet d'arrêter momentanément l'émission des données et ainsi au buffer de réception de se vider.
- b) Côte émetteur, on met en place le mécanisme dit de "**sender avoidance of silly window**". l'émetteur transmet si une des conditions suivantes est vérifiée :
 - un segment complet peut-être émis
 - il peut envoyer au moins 1/2 de la fenêtre maximum annoncée de l'autre côté (1/2 buffer du récepteur)
 - il peut envoyer ce qu'il a (PUSH positionné) et il n'y a pas de données acquittées en attente ou l'algorithme de Nagle a été désactivé

Analyse des traces TCP

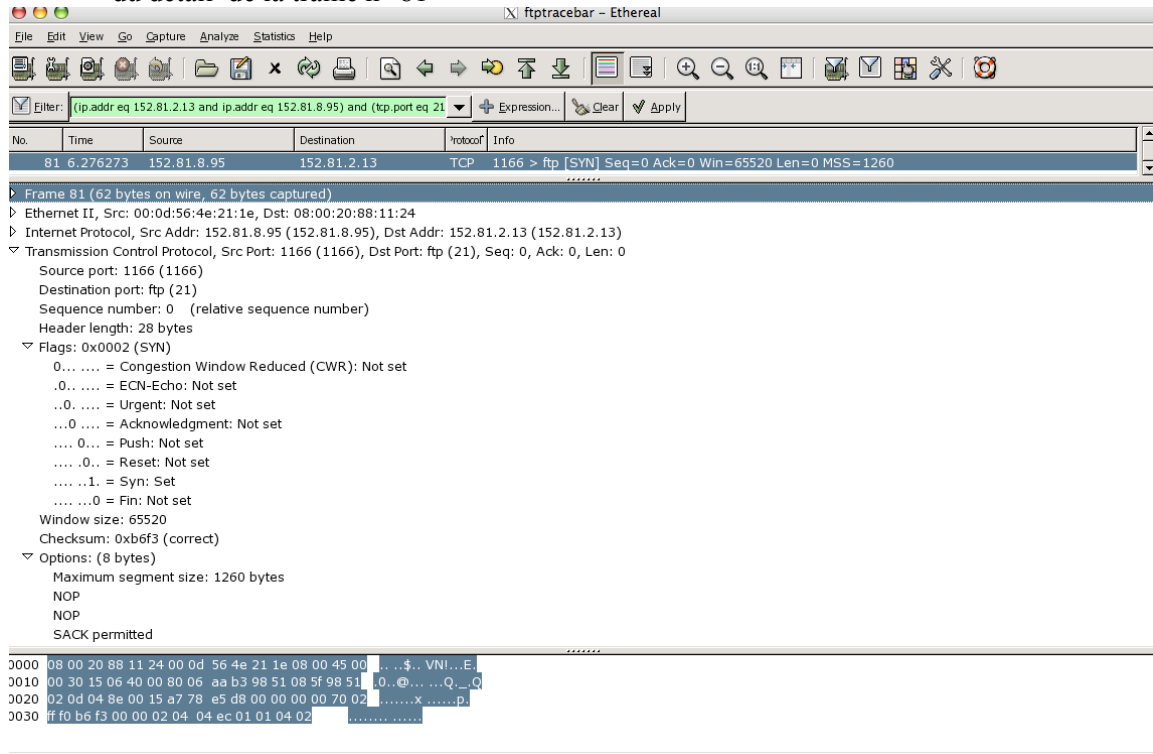
Vous disposez ci-dessous

- du résumé des échanges obtenus avec l'outil *ethereal* suite à une commande *ftp* effectuée entre 2 machines connectées à un réseau Ethernet.

No.	Time	Source	Destination	Proto.	Info
81	6.27	152.81.8.95	152.81.2.13	TCP	1166 > ftp [SYN] Seq=0 Ack=0 Win=65520 Len=0 MSS=1260
82	6.27	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [SYN, ACK] Seq=0 Ack=?? Win=8820 Len=0 MSS=1260
83	6.27	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=?? Ack=?? Win=65520 Len=0
85	6.29	152.81.2.13	152.81.8.95	FTP	Response: 220 bar.loria.fr FTP server (Version 4.116 Thu Oct 17 14:43:27 MET DST 1996) ready (Len=85)
87	6.41	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=?? Ack=?? Win=65435 Len=0
121	8.25	152.81.8.95	152.81.2.13	FTP	Request: USER ichris Len=13
122	8.25	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=86 Ack=14 Win=65520 Len=0
123	8.26	152.81.2.13	152.81.8.95	FTP	Response: 331 Password required for ichris (Len=35)
125	8.41	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=14 Ack=121 Win=65400 Len=0
2604	11.00	152.81.8.95	152.81.2.13	FTP	Request: PASS ##### (Len=15)
2605	11.04	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=121 Ack=29 Win=65520 Len=0
2659	11.09	152.81.2.13	152.81.8.95	FTP	Response: 230 User ichris logged in (Len=28)
3312	11.22	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=?? Ack=?? Win=65372 Len=0
3415	14.52	152.81.8.95	152.81.2.13	FTP	Request: PORT 152,81,8,95,4,144 (Len=24)
3416	14.52	152.81.2.13	152.81.8.95	FTP	Response: 200 PORT command successful (Len=30)
3417	14.52	152.81.8.95	152.81.2.13	FTP	Request: NLST (Len=6)
3419	14.56	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=179 Ack=59 Win=65520 Len=0
3421	14.71	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [SYN] Seq=0 Ack=0 Win=64240 Len=0 MSS=1460
3422	14.71	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [SYN, ACK] Seq=0 Ack=?? Win=65520 Len=0 MSS=1260
3423	14.71	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=?? Ack=?? Win=64260 Len=0
3424	14.71	152.81.2.13	152.81.8.95	FTP	Response: 150 Opening ASCII mode data connection for .. (Len=47)
3425	14.82	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=59 Ack=226 Win=65295 Len=0
3428	15.01	152.81.2.13	152.81.8.95	FTP	Response: 226 Transfer complete (Len=24)
3429	15.01	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3430	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3431	15.02	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [ACK] Seq=1 Ack=?? Win=65520 Len=0
3432	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3433	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1111 bytes (Len=1111)
3434	15.02	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [ACK] Seq=1 Ack=?? Win=65520 Len=0
3437	15.12	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=59 Ack=250 Win=65271 Len=0
3444	15.50	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [FIN, ACK] Seq=1 Ack=4893 Win=65520 Len=0
3445	15.50	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=?? Ack=?? Win=64260 Len=0
3508	18.61	152.81.8.95	152.81.2.13	FTP	Request: QUIT (Len=6)
3509	18.61	152.81.2.13	152.81.8.95	FTP	Response: 221 Goodbye. (Len=14)
3510	18.61	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [FIN, ACK] Seq=?? Ack=?? Win=65520 Len=0
3511	18.61	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=?? Ack=?? Win=65257 Len=0
3512	18.62	152.81.8.95	152.81.2.13	TCP	1166 > ftp [FIN, ACK] Seq=?? Ack=?? Win=65257 Len=0

3513 18.62 152.81.2.13 152.81.8.95 TCP ftp > 1166 [ACK] Seq=265 Ack=66
Win=65520 Len=0

- du détail de la trame n° 81



Questions :

1. Analyser l'en-tête TCP de la trame 81, plus particulièrement :
 - Le numéro de port destination
 - Le numéro de séquence. Quelle est sa valeur réelle dans la trace ?
 - La signification des flags. Que pouvez-vous en déduire de ce segment TCP ?
 - La taille de la fenêtre
 - La valeur des options

Il s'agit de reprendre ici avec les étudiants le détail de l'en-tête TCP et de faire la relation avec la trace en hexadécimal (le support de cours pourra être repris).

Numéro de port destination 0015 => 21 pour le protocole ftp

Numéro de séquence : car nombre relatif (sur la trace a778e5d8)

Signification des flags : Flag SYN pour demande de connexion

2. Pour les trames 82, 83, 87, 3312, 3419, 3422, 3423, 3431, 3445, 3510, 3511, 3512, compléter les champs représentés par ?? avec les véritables valeurs. Vous pouvez, pour cela, dessiner le diagramme d'échange entre le client et le serveur.
3. Comment comprenez-vous la trame 3415 ?

Le client précise au serveur le numéro de port sur lequel le serveur pourra effectuer une demande d'ouverture de connexion (trame 3421) : numéro de port = $4 \times 256 + 144 = 1168$. Il s'agit ici d'un ftp en mode actif. En général, on préfère utiliser le ftp en mode passif où c'est le client qui fait la demande de connexion vers le serveur.

81	6.27	152.81.8.95	152.81.2.13	TCP	1166 > ftp [SYN] Seq=0 Ack=0 Win=65520 Len=0 MSS=1260
82	6.27	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [SYN, ACK] Seq=0 Ack=1 Win=8820 Len=0 MSS=1260
83	6.27	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=1 Ack=1 Win=65520 Len=0
85	6.29	152.81.2.13	152.81.8.95	FTP	Response: 220 bar.loria.fr FTP server (Version 4.116 Thu Oct 17 14:43:27 MET DST 1996) ready (Len=85)
87	6.41	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=1 Ack=86 Win=65435 Len=0
121	8.25	152.81.8.95	152.81.2.13	FTP	Request: USER ichris Len=13
122	8.25	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=86 Ack=14 Win=65520 Len=0
123	8.26	152.81.2.13	152.81.8.95	FTP	Response: 331 Password required for ichris (Len=35)
125	8.41	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=14 Ack=121 Win=65400 Len=0
2604	11.00	152.81.8.95	152.81.2.13	FTP	Request: PASS ##### (Len=15)
2605	11.04	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=121 Ack=29 Win=65520 Len=0
2659	11.09	152.81.2.13	152.81.8.95	FTP	Response: 230 User ichris logged in (Len=28)
3312	11.22	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=29 Ack=149 Win=65372 Len=0
3415	14.52	152.81.8.95	152.81.2.13	FTP	Request: PORT 152,81,8,95,4,144
3416	14.52	152.81.2.13	152.81.8.95	FTP	Response: 200 PORT command successful (Len=30)
3417	14.52	152.81.8.95	152.81.2.13	FTP	Request: NLST (Len=6)
3419	14.56	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=179 Ack=59 Win=65520 Len=0
3421	14.71	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [SYN] Seq=0 Ack=0 Win=64240 Len=0 MSS=1460
3422	14.71	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [SYN, ACK] Seq=0 Ack=1 Win=65520 Len=0 MSS=1260
3423	14.71	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=1 Ack=1 Win=64260 Len=0
3424	14.71	152.81.2.13	152.81.8.95	FTP	Response: 150 Opening ASCII mode data connection for .. (Len=47)
3425	14.82	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=59 Ack=226 Win=65295 Len=0
3428	15.01	152.81.2.13	152.81.8.95	FTP	Response: 226 Transfer complete
3429	15.01	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3430	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3431	15.02	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [ACK] Seq=1 Ack=2521 Win=65520 Len=0
3432	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1260 bytes (Len=1260)
3433	15.02	152.81.2.13	152.81.8.95	FTP-DATA	FTP Data: 1111 bytes (Len=1111)
3434	15.02	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [ACK] Seq=1 Ack=4893 Win=65520 Len=0
3437	15.12	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=59 Ack=250 Win=65271 Len=0
3444	15.50	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [FIN, ACK] Seq=1 Ack=4893 Win=65520 Len=0
3445	15.50	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=4893 Ack=2 Win=64260 Len=0
3508	18.61	152.81.8.95	152.81.2.13	FTP	Request: QUIT (Len=6)
3509	18.61	152.81.2.13	152.81.8.95	FTP	Response: 221 Goodbye. (Len=14)
3510	18.61	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [FIN, ACK] Seq=264 Ack=65 Win=65520 Len=0
3511	18.61	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=65 Ack=265 Win=65257 Len=0
3512	18.62	152.81.8.95	152.81.2.13	TCP	1166 > ftp [FIN, ACK] Seq=65 Ack=265 Win=65257 Len=0
3513	18.62	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=265 Ack=66 Win=65520 Len=0

82	6.27	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [SYN, ACK] Seq=0 Ack=1
Win=8820 Len=0 MSS=1260					
83	6.27	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=1 Ack=1 Win=65520
Len=0					
87	6.41	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=1 Ack=86 Win=65435
Len=0					
3312	11.22	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=29 Ack=149
Win=65372 Len=0					
3419	14.56	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [ACK] Seq=179 Ack=59
Win=65520 Len=0					
3422	14.71	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [SYN, ACK] Seq=0 Ack=1
Win=65520 Len=0 MSS=1260					
3423	14.71	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=1 Ack=1
Win=64260 Len=0					
3431	15.02	152.81.8.95	152.81.2.13	TCP	1168 > ftp-data [ACK] Seq=1 Ack=2521
Win=65520 Len=0					
3445	15.50	152.81.2.13	152.81.8.95	TCP	ftp-data > 1168 [ACK] Seq=4893 Ack=2
Win=64260 Len=0					
3510	18.61	152.81.2.13	152.81.8.95	TCP	ftp > 1166 [FIN, ACK] Seq=264 Ack=65
Win=65520 Len=0					
3511	18.61	152.81.8.95	152.81.2.13	TCP	1166 > ftp [ACK] Seq=65 Ack=265
Win=65257 Len=0					
3512	18.62	152.81.8.95	152.81.2.13	TCP	1166 > ftp [FIN, ACK] Seq=65 Ack=265
Win=65257 Len=0					